

Claude Code CLI

Complete Setup Guide & Reference Notes

Windows • Git Bash • Node.js Environment

1 Node.js

1.1 What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside of a web browser. It is built on Google's V8 JavaScript engine and allows developers to write server-side code, CLI tools, and automation scripts in JavaScript.

Think of it this way: just as Python lets you run .py scripts from the terminal, Node.js lets you run JavaScript (.js) files from the terminal. It also bundles npm (Node Package Manager), which is the tool used to install third-party CLI tools.

1.2 Why is Node.js Needed?

Claude Code CLI is a JavaScript-based tool distributed via npm. To install or run any npm package on your computer, Node.js must be installed first. Without it:

- The npm command does not exist.
- You cannot install Claude Code CLI.
- Even if Claude Code files are copied manually, the JavaScript engine to run them is missing.

Minimum required version: Node.js v18 or higher (LTS recommended).

1.3 Check if Node.js is Already Installed

Open Git Bash and run:

```
node --version

# If installed, you will see something like:
# v20.11.0
```

If you see "command not found", proceed to install below.

1.4 How to Install Node.js

Option A — Official Installer (Recommended)

1. Go to <https://nodejs.org> in your browser.
2. Click the LTS (Long Term Support) download button.
3. Run the downloaded .msi installer.
4. Follow the wizard. Ensure "Add to PATH" is checked (it usually is by default).
5. Click Install, then Finish.

6. Restart Git Bash and verify:

```
node --version
npm --version
```

Option B — Using nvm (Advanced)

nvm (Node Version Manager) lets you install and switch between multiple Node.js versions:

```
# Run in Git Bash to install nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |
bash

# Restart Git Bash, then install Node.js LTS
nvm install --lts
nvm use --lts

# Verify
node --version
npm --version
```

Important — PATH Configuration

During the Node.js installer, make sure "Add to PATH" is checked. Without this, node and npm commands will not work in Git Bash even after installation.

2 npm — Node Package Manager

2.1 What is npm?

npm (Node Package Manager) is the default package manager bundled with Node.js. It is the world's largest software registry with hundreds of thousands of open-source packages. npm lets you install, update, and manage JavaScript tools globally or per-project.

2.2 Why is npm Needed?

Claude Code CLI is published on the npm registry under the package name `@anthropic-ai/claude-code`. npm downloads the package, sets up the executable, and makes the `claude` command available system-wide. You do not install npm separately — it comes automatically with Node.js.

2.3 Essential npm Commands

Item / Flag	Details
<code>npm --version</code>	Check npm is installed and see version
<code>npm install -g <package></code>	Install a package globally (available everywhere)
<code>npm update -g <package></code>	Update a globally installed package

<code>npm uninstall -g <package></code>	Remove a globally installed package
<code>npm list -g --depth=0</code>	List all globally installed packages

2.4 Verify npm Installation

```
npm --version

# Expected output (example):
# 10.2.4
```

2.5 Update npm to Latest (Optional)

```
npm install -g npm@latest
```

3 Shell Options, Git & Git Bash

3.1 Shell Options for Claude Code on Windows

Claude Code on Windows does not require a specific shell. You have two good options — pick the one that suits your setup:

☑ Option A — PowerShell 7 (Simplest Path)

PowerShell 7 is a fully supported shell for Claude Code on Windows and is the simplest option if your only goal is running Claude Code CLI. It is a standalone install separate from the built-in Windows PowerShell 5. Once installed, the "claude" command works directly inside it — no Git or Git Bash required. Download: <https://aka.ms/powershell> After install, open PowerShell 7 (not the older Windows PowerShell) and run: `claude`

☑ Option B — Git Bash (Good if Git is Already Installed)

Git Bash, which comes bundled with Git for Windows, provides a Unix-style terminal on Windows. It is another valid shell to run Claude Code — not the only option. If you already have Git installed, Git Bash is convenient. If not, PowerShell 7 is simpler.

The sections below cover Git and Git Bash in detail. If you choose PowerShell 7 and do not need Git for version control, you can skip ahead to Section 4 (Claude Code CLI Installation).

3.2 What is Git?

Git is the industry-standard distributed version control system. It tracks changes to files over time, allows multiple developers to collaborate, and maintains a full history of every edit. Created by Linus Torvalds in 2005, Git is used in virtually every software project worldwide.

3.3 What is Git Bash?

Git Bash is a terminal emulator for Windows that provides a Unix/Linux-style bash shell alongside Git. On macOS/Linux, bash is built in. On Windows, Git Bash fills that gap — it gives you a proper shell environment to run Unix-style commands and CLI tools like Claude Code.

3.4 Why are Git & Git Bash Needed?

- Git Bash provides the shell environment to run Claude Code CLI commands on Windows.
- Claude Code CLI works best in a Unix-style terminal (bash/zsh). Git Bash provides exactly that on Windows.
- Claude Code uses Git to read repo context, understand project structure, and assist with code changes.
- Many Claude Code workflows involve Git operations: diffs, commits, branching, and history — all powered by Git.

3.5 Check if Git is Already Installed?

```
git --version

# Expected output (example):
# git version 2.43.0.windows.1
```

3.6 How to Install Git & Git Bash

1. Go to: <https://git-scm.com/download/win>
2. Download the latest 64-bit Git for Windows installer.
3. Run the installer. Use these recommended settings during setup:

Item / Flag	Details
Default editor	Visual Studio Code (or your preferred editor)
PATH environment	"Git from the command line and also from 3rd-party software"
Line endings	"Checkout Windows-style, commit Unix-style line endings"
Terminal emulator	"Use MinTTY (the default terminal of MSYS2)"
Default branch name	main (recommended)

4. Complete installation and restart.
5. Open Git Bash from the Start Menu and verify:

```
git --version
bash --version
```

4

Claude Code CLI

4.1 What is Claude Code CLI?

Claude Code is an agentic command-line tool built by Anthropic that brings Claude's AI capabilities directly into your development workflow. Unlike a web chat interface, Claude Code runs in your terminal, has direct access to your local files and project structure, can execute commands, and works autonomously on complex multi-step tasks.

Key capabilities:

- Reads, writes, and edits files in your project directly.
- Runs shell commands and scripts on your behalf.
- Searches your codebase for context before answering.
- Integrates with Git to understand version history and diffs.
- Executes multi-step agentic workflows without constant manual input.

4.2 Why Use Claude Code CLI?

Traditional AI chat interfaces require copying and pasting code back and forth. Claude Code eliminates that by working directly inside your project. Practical use cases:

- Refactor large codebases with a single natural-language instruction.
- Debug complex issues by giving Claude direct file access.
- Automatically write tests, documentation, or boilerplate code.
- Get context-aware code reviews without leaving the terminal.
- Automate repetitive development tasks end-to-end.

4.3 Installation Steps

Step 1 — Confirm Prerequisites

Ensure Node.js (v18+), npm, and Git Bash are installed (see Sections 1–3).

```
node --version    # must be v18 or higher
npm --version     # must exist
git --version     # must exist
```

Step 2 — Install Claude Code CLI Globally

Open Git Bash and run the following command:

```
npm install -g @anthropic-ai/claude-code
```

i Why use -g (global)?

The -g flag installs Claude Code so the "claude" command is available from any directory on your system. Without -g, it would only work inside the specific folder where you ran the install command.

Step 3 — Verify Installation

```

claude --version

# Expected output:
# @anthropic-ai/claude-code/1.x.x linux-x64 node-v20.x.x

```

Step 4 — Update Claude Code (when needed)

```

npm update -g @anthropic-ai/claude-code

```

Step 5 — Uninstall (if needed)

```

npm uninstall -g @anthropic-ai/claude-code

```

4.4 Troubleshooting Installation

Item / Flag	Details
EACCES permission error	Run Git Bash as Administrator, or use nvm to manage Node.js
npm command not found	Node.js not in PATH — reinstall Node.js with PATH option checked
claude: command not found after install	Restart Git Bash so PATH refreshes
Network / 403 error	Check firewall or corporate proxy settings
Older version installed	Run <code>npm update -g @anthropic-ai/claude-code</code>

5

Anthropic API Key

5.1 What is an API Key?

An API key is a unique secret credential that identifies your account when making requests to Anthropic's Claude API. It acts like a password telling the server "this request comes from an authorized user."

5.2 Why is an API Key Needed?

Claude Code CLI sends your requests to Anthropic's cloud servers (where the Claude model runs). Every request must include your API key so Anthropic can:

- Verify your identity and account status.
- Track your usage for billing.
- Apply your account's rate limits and model permissions.

Without a valid API key, Claude Code will return an authentication error and refuse to run.

5.3 How to Get an API Key

1. Go to: <https://console.anthropic.com>

2. Sign in or create a free Anthropic account.
3. In the left sidebar, click "API Keys".
4. Click "Create API Key".
5. Name it (e.g., "Claude-Code-Local") and click Create.
6. Copy the key immediately — it is shown only once.

Security Warning

Treat your API key exactly like a password. Never put it in source code files, never commit it to Git, and never share it publicly. If leaked, revoke it immediately at console.anthropic.com and generate a new one.

6 Authentication in Git Bash

6.1 Overview

Authentication links your Claude Code installation to your Anthropic account. Without completing this step, the `claude` command will fail on every request. There are two methods available:

- Method A: Interactive browser login (easiest — recommended for most users)
- Method B: Direct API key configuration (best for servers or restricted environments)

6.2 Method A — Interactive Browser Login (Recommended)

Step 1 — Open Git Bash and start Claude Code

```
claude
```

Step 2 — Select login option

Claude Code displays an interactive selection menu. Use arrow keys and press Enter:

```
> Login with Anthropic Console (browser)    <-- select this
Enter API key directly
Exit
```

Step 3 — Authorize in the browser

Your default browser opens automatically. Log in to your Anthropic account if prompted, then click the "Authorize" button to grant Claude Code access.

Step 4 — Return to Git Bash

After clicking Authorize, return to Git Bash. You should see a success confirmation and Claude Code's interactive prompt will appear, ready to use.

```
# You should see:
✓ Authentication successful
> Type your first message...
```

6.3 Method B — Direct API Key Configuration

Use this method if browser authentication is unavailable (e.g., headless servers, corporate networks).

Option 1 — Enter API key when prompted interactively

When Claude Code shows the login menu, select "Enter API key directly" and paste your key:

```
claude

# Select: Enter API key directly
# Paste your key:
# sk-ant-api03-XXXXXXXXXXXXXXXXXXXXXXXXXXXX...
```

Option 2 — Set API key as environment variable (current session only)

This sets the key only for the current Git Bash session. You will need to repeat this each time you open a new terminal:

```
export ANTHROPIC_API_KEY="sk-ant-api03-XXXXXXXXXXXX..."
claude
```

Option 3 — Set API key permanently via ~/.bashrc (Recommended)

This permanently saves your API key so it loads automatically every time you open Git Bash:

```
# Step 1: Add the key to ~/.bashrc
echo 'export ANTHROPIC_API_KEY="sk-ant-api03-XXXXXXXXXXXX..."' >> ~/.bashrc

# Step 2: Apply changes to the current session immediately
source ~/.bashrc

# Step 3: Verify the variable is set
echo $ANTHROPIC_API_KEY

# Step 4: Start Claude Code (no key entry needed)
claude
```

About ~/.bashrc

The ~/.bashrc file is a shell configuration script that runs automatically every time you open a new Git Bash session. Adding your API key there means it is available in every session without manual entry. Replace the key value with your actual key from console.anthropic.com.

6.4 Verify Authentication is Working

After authenticating, run a quick test to confirm everything is connected:

```
claude "Hello, are you connected?"

# Claude should respond confirming the connection is live.
```

6.5 Fixing Authentication Errors

Item / Flag	Details
Invalid API key / 401 Unauthorized	Key is wrong, expired, or revoked. Generate a new one at console.anthropic.com
429 Rate limit exceeded	Usage limit reached. Wait or upgrade your Anthropic plan
ANTHROPIC_API_KEY not set	Re-run export command or source ~/.bashrc
Connection timeout	Check internet connection, firewall, or proxy settings
Authentication loop	Run <code>claude auth</code> to reset and re-authenticate

7 First Use & Essential Commands

7.1 Starting Claude Code

```
# Navigate to your project first
cd /path/to/your/project

# Start interactive session
claude

# One-off question (non-interactive)
claude "Explain what this project does"
```

7.2 Useful CLI Flags

Item / Flag	Details
<code>claude --help</code>	Show all available commands and options
<code>claude --version</code>	Show installed Claude Code version
<code>claude --print</code>	Print Claude's response without entering interactive mode
<code>claude --no-auto-update</code>	Disable automatic version updates
<code>claude --model <name></code>	Specify which Claude model to use

7.3 In-Session Slash Commands

Once inside an active Claude Code session, use these commands:

Item / Flag	Details
<code>/help</code>	Show all slash commands available in the session
<code>/clear</code>	Clear the conversation history to start fresh
<code>/compact</code>	Compress the conversation to save context window space

<code>/cost</code>	Show token usage and estimated cost for this session
<code>/doctor</code>	Check installation health and environment status
<code>/quit</code> or <code>/exit</code>	Exit Claude Code and return to shell

7.4 Example: Full Workflow from Start to Finish

```
# 1. Navigate to your project
cd ~/projects/my-app

# 2. Start Claude Code
claude

# 3. Ask Claude to understand the project
> Summarize the structure and purpose of this codebase.

# 4. Ask Claude to make a code change
> Add input validation to the login form in src/auth/login.js

# Claude shows you the planned changes (diff) and asks for approval
# before writing anything to disk.

# 5. Exit when done
> /exit
```

8

Complete Setup Checklist

Use this checklist to confirm your Claude Code CLI environment is fully operational:

#	Task	Verify With
1	Install Node.js v18 or higher	<code>node --version</code>
2	Confirm npm is available	<code>npm --version</code>
3	Install Git and Git Bash	<code>git --version</code>
4	Install Claude Code CLI globally via npm	<code>claude --version</code>
5	Obtain Anthropic API Key from console.anthropic.com	(web step)
6	Authenticate Claude Code (browser or API key)	<code>claude "hello"</code>
7	Confirm Claude responds correctly	<code>claude --help</code>

 **All Done!**

Once all 7 steps are complete, your Claude Code CLI environment is fully operational. Open Git Bash in any project directory and type "claude" to start an AI-powered coding session.